

FEATURE SCALING

Feature scaling is a preprocessing technique used to transform the numerical features of a dataset to a standard, common scale. We do this to prevent any single feature with a naturally large range (like 'Income' in the millions) from unfairly dominating or overshadowing other important features (like 'Age' in the tens) during model training, ensuring all features contribute equally to the final prediction."

In machine learning, many algorithms use distance, gradient calculations, or weight updates. If features have vastly different ranges—like age (0–100) and income (0–100,000)—the algorithm may become biased toward features with larger values. Feature scaling normalizes these differences by converting all numerical features to a similar scale. This allows models to learn more efficiently, converge faster during training, and treat all features fairly without unintended bias due to magnitude differences.

Feature Scaling Methods

1. Min–Max Scaling (Normalization)

Definition:

Min–max scaling transforms each feature to a fixed range, typically $[0,1]$, using the formula

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Details:

- Preserves the shape of the original distribution.
- Sensitive to outliers (extreme values stretch the range).
- Commonly used in **neural networks**, **KNN**, **K-Means**, and any algorithm that uses distances.

2. Standardization (Z-score Scaling)

Definition:

Standardization transforms a feature so that it has **mean = 0** and **standard deviation = 1**:

$$x' = \frac{x - \mu}{\sigma}$$

Details:

- Centers data around zero and normalizes variance.
- Less sensitive than min–max to outliers (but still affected).
- Very common for **linear models**, **logistic regression**, **SVM**, **PCA**, and many deep learning models.

3. Robust Scaling

Definition:

Robust scaling uses statistics that are **robust to outliers**, such as the **median** and **interquartile range (IQR)**:

$$x' = \frac{x - \text{median}}{IQR}$$

Details:

- Median and IQR are not affected much by extreme values.
- Ideal when the data contains **outliers** that you do not want to remove but also do not want to dominate the scaling.
- Often used as a safer alternative to standardization in noisy, real-world datasets.

4. MaxAbs Scaling

Definition:

MaxAbs scaling scales features by their maximum absolute value, mapping them into $[-1,1]$:

$$x' = \frac{x}{|x_{\max}|}$$

Details:

- Preserves sparsity (important for **sparse matrices**, e.g., text data with many zeros).
- Does not shift/center data — only rescales.
- Useful when features are already centered or when centering would destroy sparsity.

5. Unit Vector Scaling (Normalizer)

Definition:

Unit vector scaling (L1/L2 normalization) rescales each **sample (row)** so that its vector norm is 1:

$$\|x'\| = 1$$

Details:

- Applied per **sample**, not per feature.
- Common in **text mining** (TF-IDF vectors) where direction matters more than magnitude.
- Very useful when using **cosine similarity** or clustering high-dimensional vectors.

6. Quantile Transformation (Uniform or Normal)

Definition:

Quantile-based scaling transforms data so that its **empirical cumulative distribution** matches a desired target distribution (e.g., uniform or normal).

Details:

- Makes the feature distribution more “regular” (e.g., approximately Gaussian).
- Very robust to outliers and non-linear relationships.
- Useful for algorithms that assume normally distributed features, or when you want all features to have similar empirical distributions.

7. Power Transformation (Yeo–Johnson / Box–Cox)

Definition:

Applies a **power transformation** (non-linear) to make data more Gaussian-like and stabilize variance.

Details:

- Reduces skewness in highly skewed features.
- Box–Cox works for strictly positive data; Yeo–Johnson works with zero/negative values as well.

- Useful before applying models that benefit from normality and stable variance (e.g., linear models, PCA).

Why Feature Scaling is Important

Feature scaling is crucial for two main categories of machine learning algorithms:

A. Distance-Based Algorithms

Algorithms that rely on measuring the distance between data points (like Euclidean Distance) will have their performance severely degraded if features are unscaled.

- **Affected Algorithms:** k -Nearest Neighbors (k -NN), k -Means Clustering, Support Vector Machines (SVMs).
- **Reason:** Unscaled features with larger magnitudes will dominate the distance calculation, effectively masking the contribution of features with smaller ranges. Scaling ensures all features contribute proportionally to the final distance.

B. Gradient Descent-Based Algorithms

Algorithms that use an iterative optimization process called **Gradient Descent** benefit greatly from scaling.

- **Affected Algorithms:** Linear Regression, Logistic Regression, Neural Networks, Gradient Boosting Machines (in some implementations).
- **Reason:** If features are on widely different scales, the **Loss Function** (the surface being optimized) will be highly elongated and asymmetrical. This forces the gradient descent optimizer to take many small, inefficient steps, leading to slow convergence and oscillation. Scaling ensures the loss surface is more spherical, allowing the optimizer to converge to the minimum much faster.

Feature scaling is essential in machine learning for several reasons. First, it improves the convergence speed of gradient-based algorithms such as logistic regression, linear regression, and neural networks. Second, it ensures that distance-based models like k -Nearest Neighbors and k -Means treat all features fairly. Third, scaling helps stabilize numerical computations, reducing the risk of exploding gradients and inconsistent model behavior. Finally, it can significantly enhance overall generalization by preventing certain features from dominating others due to scale differences.